

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Technical Presentation

Weightage: 20%

QUESTIONS:

TotalMarks:20

1.Servlet Architecture & Workflow

Question:

Present the **architecture of Java Servlets** and explain the complete **request–response cycle** with a neat diagram.

Focus Areas:

- Client–Server interaction
- Role of Web Container
- Request and Response objects

2. Servlet Life Cycle Analysis

Question:

Explain the **Servlet Life Cycle** with a sequence diagram and analyze the role of `init()`, `service()`, and `destroy()` methods.

Focus Areas:

- Life cycle phases
- Execution flow
- Performance implications

3. GET vs POST Method Evaluation

Question:

Compare **GET and POST methods** used in form submission. Present scenarios where each method is appropriate.

Focus Areas:

- Data transmission
- Security aspects
- URL limitations

4. Session Tracking Techniques**Question:**

Present different **session tracking mechanisms** (HttpSession, Cookies, URL rewriting). Evaluate their advantages and limitations.

Focus Areas:

- Session persistence
- Security concerns
- Use-case comparison

5. Cookies vs Sessions – Critical Analysis**Question:**

Analyze and compare **Cookies and Sessions** in web applications. Present real-time use cases.

Focus Areas:

- Storage location
- Security risks
- Performance considerations

Code of Conduct:

I GOUTHAM KORNANA/192311169 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:k.goutham

ANSWERS:

1. Servlet Architecture & Workflow

Question:

Present the architecture of Java Servlets and explain the complete request–response cycle with a neat diagram.

Answer:

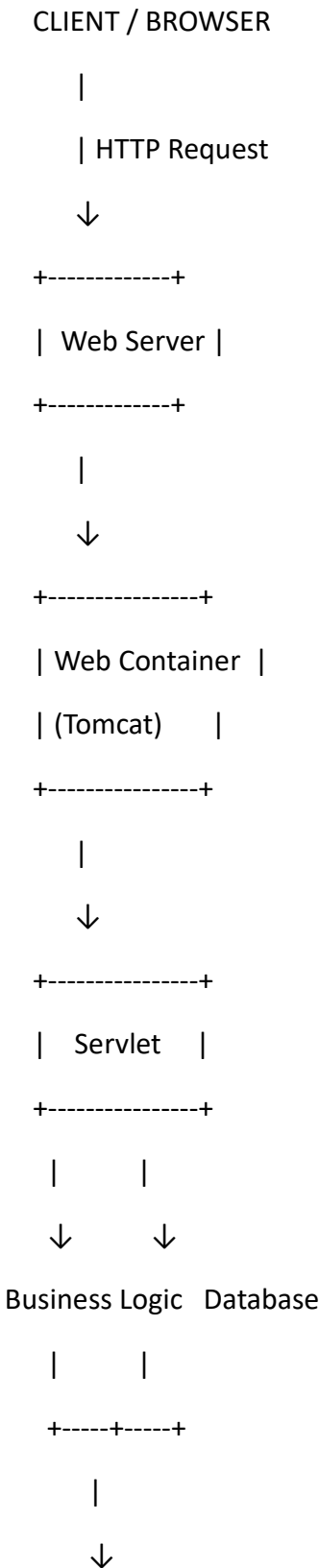
A **Java Servlet** is a server-side Java program used to create dynamic web applications. It runs inside a **Web Container**, such as Apache Tomcat. The servlet receives requests from clients, processes them, interacts with databases or other resources, and generates responses.

Servlet Architecture

The main components of servlet architecture are:

1. **Client/Browser** – Sends HTTP requests to the web server.
2. **Web Server** – Receives HTTP requests and forwards servlet-related requests to the web container.
3. **Web Container** – Manages servlet execution, lifecycle, security, and request/response objects.
4. **Servlet** – Processes the request and generates the response.
5. **Database/Business Logic** – Provides required data or performs application operations.
6. **HTTP Response** – The processed result is returned to the client.

Architecture Diagram



HTTP Response



CLIENT / BROWSER

Request–Response Cycle

The servlet request–response cycle occurs as follows:

Step 1: Client Request

The user enters a URL or submits an HTML form. The browser sends an HTTP request to the server.

Step 2: Web Server Receives Request

The web server receives the request and determines whether it should be handled by a servlet.

Step 3: Web Container Processes Request

The web container identifies the appropriate servlet using the URL mapping.

Step 4: Servlet Execution

If the servlet has not been loaded, the container loads it and initializes it using the `init()` method. The request is then processed through the `service()` method.

Step 5: Request Object

The `HttpServletRequest` object contains information such as:

- Form data
- Request parameters
- HTTP headers
- Cookies
- Session information

Step 6: Processing

The servlet performs business logic and may communicate with a database.

Step 7: Response Object

The `HttpServletResponse` object is used to create the response sent back to the browser.

Step 8: Client Receives Response

The browser receives the HTTP response and displays the generated HTML or other content.

Example

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class HelloServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                           HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<h1>Hello from Servlet</h1>");
    }
}
```

Conclusion

Servlet architecture provides an efficient mechanism for developing dynamic web applications. The **web container** manages servlet lifecycle and communication between the client and servlet, while `HttpServletRequest` and `HttpServletResponse` handle request and response data.

2. Servlet Life Cycle Analysis

Question:

Explain the Servlet Life Cycle with a sequence diagram and analyze the role of `init()`, `service()`, and `destroy()` methods.

Answer:

The **Servlet Life Cycle** describes the stages through which a servlet passes from creation to destruction. The web container manages the complete lifecycle.

The major stages are:

1. Loading the servlet
2. Creating servlet object
3. Calling `init()`
4. Calling `service()`
5. Calling `destroy()`

Servlet Life Cycle Diagram

Servlet Class

|

↓

Load Servlet Class

|

↓

Create Servlet Object

|

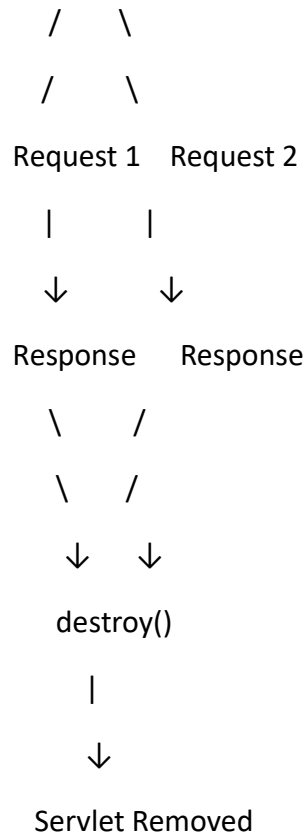
↓

`init()`

|

↓

`service()`



1. init() Method

The `init()` method is called **only once** when the servlet is initialized.

It is mainly used for:

- Initializing resources
- Reading configuration
- Establishing database connections
- Performing one-time setup operations

Example:

```
public void init() throws ServletException {  
    System.out.println("Servlet Initialized");  
}
```


2. service() Method

The service() method is called for **each client request**.

For an HttpServlet, it determines whether the request is a GET, POST, PUT, DELETE, etc., and dispatches it to methods such as doGet() or doPost().

```
protected void doGet(HttpServletRequest request,  
                    HttpServletResponse response)  
    throws IOException {  
  
    response.getWriter().println("GET Request Received");  
}
```

3. destroy() Method

The destroy() method is called **once before the servlet is removed** from service.

It is used to:

- Close database connections
- Release resources
- Perform cleanup operations

```
public void destroy() {  
  
    System.out.println("Servlet Destroyed");  
}
```

Sequence of Execution

Container

```
|  
|-- Load Servlet  
|  
|-- Create Object  
|  
|-- init()  
|  
|-- service()  
|  |  
|  --> doGet()/doPost()  
|  
|-- service()  
|  |  
|  --> doGet()/doPost()  
|  
|-- destroy()
```

Performance Implications

Servlets improve performance because a servlet object is generally created once and can handle multiple requests. Creating a new servlet object for every request would consume more memory and processing time.

However, shared servlet resources must be handled carefully because multiple requests can execute concurrently.

Conclusion

The servlet lifecycle is controlled by the web container. `init()` performs one-time initialization, `service()` processes requests, and `destroy()` performs cleanup. Proper lifecycle management improves application performance and resource utilization.

3. GET vs POST Method Evaluation

Question:

Compare GET and POST methods used in form submission. Present scenarios where each method is appropriate.

Answer:

GET and POST are HTTP methods commonly used for sending data from a client to a server.

Comparison

Feature	GET	POST
Data location	URL	Request body
Visibility	Data visible in URL	Data not normally displayed in URL
Data size	Limited by URL length	Supports larger data
Bookmarking	Can be bookmarked	Generally not suitable
Caching	Can be cached	Usually not cached by default
Passwords	Not appropriate	Preferable, but HTTPS is required
Main purpose	Retrieve data	Submit/change data
Idempotency	Generally safe/idempotent	May create/change server state

GET Example

HTML form:

```
<form action="LoginServlet" method="get">
```

```
  <input type="text" name="username">
```

```
  <input type="submit" value="Submit">
```

```
</form>
```

Servlet:

```
protected void doGet(HttpServletRequest request,
    HttpServletResponse response)
    throws IOException {

    String name = request.getParameter("username");

    response.getWriter().println("Welcome " + name);
}
```

The resulting request may look like:

```
/LoginServlet?username=Sai
```

POST Example

```
<form action="LoginServlet" method="post">
    <input type="text" name="username">
    <input type="password" name="password">
    <input type="submit" value="Login">
</form>
```

Servlet:

```
protected void doPost(HttpServletRequest request,
    HttpServletResponse response)
    throws IOException {

    String username = request.getParameter("username");
    String password = request.getParameter("password");
```

```
response.getWriter().println("Login request received");  
}
```

When to Use GET

GET is appropriate when:

- Retrieving information
- Searching products
- Filtering results
- Accessing a webpage
- Sharing/bookmarking a result

Example:

/products?category=laptop

When to Use POST

POST is appropriate when:

- Creating an account
- Login submission
- Uploading data
- Submitting a feedback form
- Creating or modifying server-side data

Security Analysis

POST is generally preferable for sensitive form data because the data is sent in the request body rather than the URL. However, **POST alone does not encrypt data**. HTTPS should be used for passwords, personal information, payment information, and other sensitive data.

Conclusion

GET is mainly suitable for **retrieving data**, while POST is suitable for **submitting or modifying data**. Choosing the correct method improves security, usability, and application design.

4. Session Tracking Techniques

Question:

Present different session tracking mechanisms (HttpSession, Cookies, URL rewriting). Evaluate their advantages and limitations.

Answer:

HTTP is a **stateless protocol**, meaning the server does not automatically remember previous requests from the same user. Session tracking mechanisms are used to maintain user information across multiple requests.

The major techniques are:

1. HttpSession
2. Cookies
3. URL Rewriting

1. HttpSession

HttpSession stores session-related information on the server.

Example

```
HttpSession session = request.getSession();
```

```
session.setAttribute("username", "Sai");
```

To retrieve:

```
String name =
```

```
(String) session.getAttribute("username");
```

Advantages

- Data is stored on the server.
- Easy to use.
- Suitable for login sessions and shopping carts.
- Can store multiple attributes.

Limitations

- Consumes server memory.
 - Requires session management.
 - Session hijacking can be a security concern.
-

2. Cookies

Cookies are small pieces of data stored by the user's browser.

Example

```
Cookie cookie =
```

```
    new Cookie("username", "Sai");
```

```
response.addCookie(cookie);
```

Reading the cookie:

```
Cookie[] cookies = request.getCookies();
```

```
for (Cookie c : cookies) {
```

```
    if (c.getName().equals("username")) {
```

```
        System.out.println(c.getValue());
```

```
    }
```

```
}
```

Advantages

- Simple to implement.
- Can persist across requests.
- Reduces server-side storage for small pieces of information.

Limitations

- Users can disable cookies.

- Cookies can be stolen if security controls are weak.
- Limited storage capacity.
- Sensitive information should not be stored directly in cookies.

3. URL Rewriting

URL rewriting adds session information to the URL.

Example:

profile.jsp;jsessionid=ABC123

In Java:

```
String url =  
    response.encodeURL("profile.jsp");
```

Advantages

- Works when cookies are disabled.
- Easy fallback mechanism.

Limitations

- Session identifiers may appear in URLs.
- URLs become longer.
- URLs may be stored in browser history or logs.
- Less convenient than cookies or HttpSession.

Comparison

Feature	HttpSession	Cookies	URL Rewriting
Storage	Server	Client	URL
Security	Relatively better	Depends on configuration	Lower
Server memory	Required	Low	Required

Feature	HttpSession	Cookies	URL Rewriting
Works without cookies	Yes	No	Yes
Ease of use	High	High	Moderate
Common use	Login/cart	Preferences	Cookie fallback

Conclusion

HttpSession is generally preferred for server-side session management. Cookies are useful for small client-side values and session identifiers, while URL rewriting can be used as an alternative when cookies are unavailable.

5. Cookies vs Sessions – Critical Analysis

Question:

Analyze and compare Cookies and Sessions in web applications. Present real-time use cases.

Answer:

Cookies and sessions are both used to maintain state in web applications. The main difference is **where the information is stored**.

Cookies

Cookies are small pieces of data stored on the user's browser.

Example:

Browser

|

| Cookie

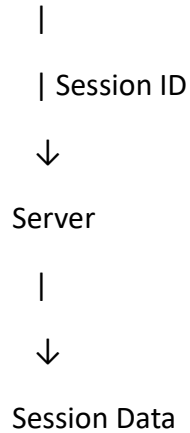
↓

username = Sai

Sessions

Sessions store user-specific information on the server. The browser generally carries a session identifier, commonly through a cookie.

Browser

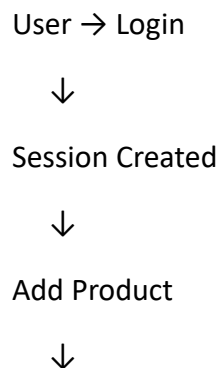


Cookies vs Sessions

Feature	Cookies	Sessions
Storage location	Client/browser	Server
Data exposure	More exposed to client	Data remains server-side
Capacity	Small	Depends on server resources
Server memory	Low	Higher
Persistence	Can be persistent	Usually temporary
Security	Can be vulnerable to theft/manipulation	Generally safer for sensitive state
Performance	Reduces server storage	Requires server-side storage
Common use	Preferences, tracking	Login, shopping cart

Real-Time Use Case 1: Online Shopping

A shopping website can use sessions to maintain a user's cart:



Session stores Cart



Checkout

This prevents the cart from being lost when the user moves between pages.

Real-Time Use Case 2: Remembering Preferences

Cookies can store non-sensitive preferences such as:

language = English

theme = dark

When the user returns to the website, these preferences can be read by the application.

Real-Time Use Case 3: Login Authentication

After successful login, a session can maintain the user's authenticated state.

Login



Validate Credentials



Create Session



Session ID



Access Protected Pages

Security Considerations

For session security:

- Use HTTPS.
- Use secure session identifiers.
- Configure cookies with Secure and HttpOnly where appropriate.
- Use SameSite settings to reduce certain cross-site risks.

- Expire inactive sessions.
- Regenerate the session identifier after authentication.

For cookies:

- Avoid storing passwords or sensitive information directly.
- Use HttpOnly to reduce JavaScript access when appropriate.
- Use Secure when transmitting over HTTPS.
- Set an appropriate SameSite policy.

Performance Considerations

Cookies are sent with applicable HTTP requests, increasing request size. Sessions require server-side resources, especially when many users are simultaneously active.

Therefore, applications should store only necessary information in sessions and avoid unnecessarily large cookies.

RUBRICS FOR CALCULATION:

Technical Presentation					
Criteria	Weight	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood

Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism ; minimal contribution or ethical awareness